Instituto Politécnico Nacional
Centro de Investigación en Computación

Diseño y Análisis de Algoritmos

Nombre Completo: Mondragón Vega Fernando Iván
Fecha de entrega: 5 de Octubre de 2026

TAREA No. 2

1. Suppose you have algorithms with the six running times listed below. (Assume these are the exact number of operations performed as a function of input size n.) Suppose you have a computer that can perform 10^{10} operations per second, and you need to compute a result in at most an hour of computation. For each of the algorithms, what is the largest input size n for which you would be able to get the result within an hour?

   a) n^2
   b) n^3
   c) $100n^2$
   d) $n \log n$
   e) 2^n
   f) 2^{2^n}

   Solution
   For each function $f(n)$, we need to find the maximum natural number n_0 so that: $f(n_0) \leq 3600 \times 10^{10}$.

   a)

$$\begin{aligned} n^2 &\leq 3600 \times 10^{10} \\ n &\leq \sqrt{3600 \times 10^{10}} \\ n &\leq 60 \times 10^5 \\ n &= 6 \times 10^6 \end{aligned}$$

   b)

$$\begin{aligned} n^3 &\leq 3600 \times 10^{10} \\ n &\leq \sqrt[3]{3600 \times 10^{10}} \\ n &= \lfloor 33019{,}27 \rfloor \\ n &= 33019 \end{aligned}$$

   c)

$$\begin{aligned} 100n^2 &\leq 3600 \times 10^{10} \\ n^2 &\leq 36 \times 10^{10} \\ n &= 6 \times 10^5 \end{aligned}$$

d) We have $n \log n \leq 3600 \times 10^{10}$. To solve this inequality, we apply numerical methods. According to Newton-Raphson, $n \approx 1{,}3 \times 10^{12}$.

e)

$$\begin{aligned} 2^n &\leq 3600 \times 10^{10} \\ n &\leq \log_2(3600 \times 10^{10}) \\ n &= \lfloor 45{,}03 \rfloor \\ n &= 45 \end{aligned}$$

f)

$$\begin{aligned} 2^{2^n} &\leq 3600 \times 10^{10} \\ 2^n &\leq \log_2\left(3600 \times 10^{10}\right) \\ n &\leq log_2\,(45{,}03) \\ n &= \lfloor 5{,}49 \rfloor \\ n &= 5 \end{aligned}$$

2. Consider the following basic problem. You're given an array A consisting of n integers $A[1], A[2], ..., A[n]$. You'd like to output a two-dimensional n-by-n array B in which $B[i, j]$ (for $i < j$) contains the sum of array entries $A[i]$ through $A[j]$ - that is, the sum $A[i] + A[i+1] + ... + A[j]$. (The value of array entry $B[i, j]$ is left unspecified whenever $i \geq j$, so it doesn't matter what is output for these values.)
Here is a simple algorithm to solve this problem.

for $i = 1, 2, ..., n$ **do**
  for $j = i + 1, i + 2, ..., n$ **do**
    Add up array entries $A[i]$ through $A[j]$
    Store the result in $B[i, j]$
  end for
end for

a) For some function f that you should choose, give a bound of the form $O(f(n))$ on the running time of this algorithm on an input of size n (*i.e.*, a bound on the number of operations performed by the algorithm).

b) For this same function f, show that the running time of the algorithm on an input of size n is also $\Omega(f(n))$. (This shows an asymptotically tight bound of $\Theta(f(n))$ on the running time.)

c) Although the algorithm you analyzed in parts (a) and (b) is the most natural way to solve the problem - after all, it just iterates through the relevant entries of the array B, filling in a value for each - it contains some highly unnecessary sources of inefficiency. Give a different algorithm to solve this problem, with an symptotically better running time. In other words, you should design an algorithm with running time $O(g(n))$, where $\lim_{n\to\infty} \dfrac{g(n)}{f(n)} = 0$.

Solution

a) Claim. This algorithm has a running complexity of $O(n^3)$.
Proof. The first loop runs for n iterations. The second runs at most $n-1$ iterations, for n sufficiently large $n-1 \approx n$. Adding entries $A[i]$ through $A[j]$ takes $j-i+1$ steps, and at most n steps (for $i=1, j=n$). Storing entries takes constant time. Since this addition is repeated at worst n^2 times, this algorithm is $O(n^3)$.

b) We have the lowest iterations when i is small in comparison to n, and j is close to n. Arbitrarily let's choose $i \leq \frac{n}{10}$ and $j \geq \frac{9n}{10}$. So the addition operation is repeated $\left(\frac{n}{10}\right)^2$ times. Each one takes at least $j-i+1 = \frac{9n}{10} - \frac{n}{10} + 1 = \frac{4n}{5} + 1$ steps (the fewest number of elements between the values of i and j). In total we have $\left(\frac{n}{10}\right)^2 \frac{4n}{5} = \frac{n^3}{125}$, which is just $\Omega(n^3)$.

c) Consider the following algorithm:

Initialize B as an n x n array
$B[1][1] \leftarrow A[1]$
for $k = 2, 3, ..., n$ **do**
    $B[1][k] \leftarrow B[1][k-1] + A[k]$
end for
for $i = 2, 3, ..., n$ **do**
    for $j = i+1, i+2, ..., n$ **do**
        $B[i][j] \leftarrow B[i-1][j] - A[i-1]$
    end for
end for

Claim. This algorithm solves the problem. (By induction)

We need to prove that for any $1 \leq i < j \leq n$, $B[i][j] = A[i] + A[i+1] + ... + A[j]$. Given the nature of the algorithm, we use mathematical induction on the row index.

Demostración. (By induction)
Base case.
$B[1][1]$ is assigned to be $A[1]$
For every $k \leq n$, the algorithm assigns $B[1][k] \leftarrow B[1][k-1] + A[k]$
By expansion we get

$$B[1][k] = A[1] + A[2] + ... + A[k-1] + A[k]$$

Thus the first row is assigned correctly.
Inductive Step.
Inductive Hypotesis. We assume that for some row $k < n$, every entry is correctly assigned, so that $B[k][j] = A[k] + A[k+1] + ... + A[j-1] + A[j]$ for all $k < j \leq n$.
We need to show that row $k+1$, the algorithm correctly assigns every entry.

The algorithm assigns $B[k+1][j] \leftarrow B[k][j] - A[k]$. (For every $k < j \leq n$)
If we substitute our inductive hypotesis, we get

$$B[k+1][j] = (A[k] + A[k+1] + ... + A[j-1] + A[j]) - A[k]$$
$$B[k+1][j] = A[k+1] + ... + A[j-1] + A[j],$$

which is the correct assignment for this entry.

□

Claim. This algorithm runs in $O(n^2)$.

Proof. The first loop runs for n iterations. The second loop is not associated with the first one; it runs for $n-1 \approx n$ iterations (for n sufficiently large). The third loop runs at most $n-2 \approx n$ iterations. The assignation to entry $B[i][j]$ is done in constant time. Since this assignation is repeated at worst n^2 times, this algorithm is $O(n^2)$.

3. You're doing some stress-testing on various models of glass jars to determine the height from which they can be dropped and still not break. The setup for this experiment, on a particular type of jar, is as follows. You have a ladder with n rungs, and you want to find the highest rung from which you can drop a copy of the jar and not have it break. We call this the *highest safe rung*.
It might be natural to try binary search: drop a jar from the middle rung, see if it breaks and then recursively try from rung $n/4$ or $3n/4$ depending on the outcome. But this has the drawback that you could break a lot of jars in finding the answer.
If your primary goal were to conserve jars, on the other hand, you could try the following strategy. Start by dropping a jar from the first rung, then the second rung, and so forth, climbing one higher each time until the jar breaks. In this way, you only need a single jar - at the moment it breaks, you have the correct answer - but you may have to drop it n times (rather than $\log n$ as in binary search solution).
So here is the trade-off: it seems you can perform fewer drops if you're willing to break more jars. To understand better how this trade-off works at a quantitative level, let's consider how to run this experiment given a fixed "budget" of $k \geq 1$ jars. In other words, you have to determine the correct answer - the highest safe rung - and can use at most k jars in doing so.

   a) Suppose you are given a budget of $k = 2$ jars. Describe a strategy for finding the highest safe rung that requires you to drop a jar at most $f(n)$ times, for some function $f(n)$ that grows slower than linearly. (In other words, it should be the case that $\lim_{n\to\infty} \frac{f(n)}{n} = 0$.)

   b) Now suppose you have a budget of $k > 2$ jars, for some given k. Describe a strategy for finding the highest safe rung using at most k jars. If $f_k(n)$ denotes the number of times you need yo drop a jar according to your strategy, the the

functions $f_i, f_2, f_3, ...$ shoudl have the property that each grows asymptotically slower than the previous one: $\lim_{n\to\infty} \frac{f_k(n)}{f_{k-1}(n)} = 0$ for each k.

Solution

a) Consider the following algorithm:

$i \leftarrow \lceil \sqrt{n} \rceil$
for $k = 1, 2, ..., i$ **do**
  if $k = i$ **then**
    Throw the first jar from rung n
  else
    Throw the first jar from rung $k * i$
  end if
  if the jar breaks **then**
    Apply the algorithm for 1 jar, starting from rung $(k-1) * i + 1$
  end if
end for
return n

Claim. This algorithm finds the highest safe rung.

Proof (by contradiction). Let J be the rung returned by the algorithm and H be the actual highest safe rung. Let's assume $J \neq H$. Therefore, we have two cases.

Case 1. $J > H$
$\rightarrow J$ is unsafe.
But the algorithm returned J because the second jar was checked at position $J+1$ and the jar broke.
$\rightarrow$ The algorithm checked at position J and the jar didn't break.
$\rightarrow J$ is safe. A contradiction.

Case 2. $J < H$
$\rightarrow J + 1 \leq H$
$\rightarrow J + 1$ is safe.
Once again, the algorithm returned J because the second jar was checked at position $J + 1$ and the jar broke.
$\rightarrow J + 1$ is unsafe. A contradiction.

Since our assumption that $J \neq H$ ended up in a contradiction in both cases, that implies $J = H$. In other words, the algorithm finds the highest safe rung. □

Claim. This algorithm has a running complexity of $O(\sqrt{n})$.

Proof. Since the first loop stops when $k = i$, then it runs at most for $\lceil\sqrt{n}\rceil$ iterations. Thus, $\sqrt{n} \leq k < \sqrt{n} + 1$. So we have an upper bound of $\sqrt{n} + 1$. Then if the first jar doesn't break at the last iteration, we are done. Otherwise, we now start dropping the second jar between $(k-1)\sqrt{n} + 1$ and $k\sqrt{n}$. This interval clearly has a size of $\sqrt{n}$, at worst. The algorithm for 1 jar runs at worst z iterations, being z the length of the interval. Therefore, in the worst case we check $(\sqrt{n} + 1) + (\sqrt{n} - 1) = 2\sqrt{n}$ rungs. Which gives us a running complexity of $O(\sqrt{n})$. □

b) We follow a similar approach to last algorithm, though we make some changes. First of all, we start by checking the rungs in multiples of $\lceil n^{(k-1)/k}\rceil$. We have at most $\lceil \sqrt[k]{n}\rceil < \sqrt[k]{n} + 1$ iterations. If the jar doesn't break at the last iteration, we are done. Otherwise we take the last rung where the jar didn't break and apply the same algorithm for $k - 1$ jars, recursively. Note that our search is now limited to a range of length $\sqrt[k]{n}$. By induction, for $k - 1$ jars, we check every $\lceil \left(n^{1/k}\right)^{((k-1)-1)/(k-1)}\rceil = \lceil \left(n^{1/k}\right)^{(k-2)/(k-1)}\rceil = \lceil n^{(k-2)/k(k-1)}\rceil$ rungs, of which there are at most $\lceil \sqrt[k-1]{n^{1/k}}\rceil < n^{1/k(k-1)} + 1$ iterations. Those are added to the ones previously made. We repeat this process until we are left with one jar. In that case, we apply the algorithm previously described. Since $n^{1/k(k-1)} + 1 < n^{1/k}$ then the first loop dominates over the others and we have a complexity of $O(n^{1/k})$.